

Lab 08 — Eight Ways to Search

DSA27 Lab Manual · Week 8 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 8

Contents

Part 0 — Before you start	2
Part 1 — <code>linear_search</code> : the baseline	3
Part 2 — <code>binary_search</code> : halve what is left	4
Part 3 — Break it: <code>lo < hi</code>	6
Part 4 — <code>binary_search_recursive</code>	7
Part 5 — <code>lower_bound</code> and <code>upper_bound</code> : one loop, one comparison	8
Part 6 — Jump, exponential and interpolation search	10
Part 7 — Count the reads	13
Part 8 — Check your bounds against <code>bisect</code>	15
Part 9 — Measure it	15
Part 10 — Exercises at a glance	16
Part 11 — Take-home practice (not graded)	16
Part 12 — Bridge to Lecture 09: sorting	17
Summary	18
Answers to the checkpoints	19

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — here that means a table of `lo`, `hi` and `mid` on paper — and predict at each **Checkpoint** (answers at the end). Then write one function, run the tests named in its part, and only then move on. This is the midterm week, so the lab is kept tight: eight short functions, and one idea — a sorted array can be searched without looking at most of it. Parts 1–5 are the session; Parts 6–9 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/searching.py</code> — <code>linear_search</code> , <code>binary_search</code> , <code>binary_search_recursive</code> , <code>lower_bound</code> , <code>upper_bound</code> , <code>jump_search</code> , <code>exponential_search</code> , <code>interpolation_search</code> ; and, for yourself, a small <code>CountingReads</code> class
Graded by	<code>tests/test_searching.py</code> (56 tests)
Connects to	Lecture 08 — Searching; and Lecture 09, where sorting makes these searches possible

What you will be able to do

1. trace a binary search as a table of `lo`, `hi` and `mid`, for a hit and for a miss, and say where `lo` ends up;
2. write `binary_search` from its invariant, and test it by hand on sizes 0, 1 and 2 before running `pytest`;
3. explain exactly which tests a `lo < hi` bug fails, and why the others pass;
4. write the recursive version without slicing, and say what a slice costs;
5. write `lower_bound` and `upper_bound` as one half-open loop that differs in a single comparison, and use them to count duplicates;
6. write `jump`, `exponential` and `interpolation` search, including their edge cases;
7. count the reads each search makes with a sequence class of your own, and show interpolation search collapsing on skewed data.

Part 0 — Before you start

0.1 Environment

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_searching.py -q     # 56 failures: nothing written yet
```

The 56 failures are all `NotImplementedError` — that is the starting point.

0.2 Prerequisites

None from `dsa/`. Unlike the stack and the queues, this week's functions stand on nothing you built: they take an ordinary Python list as **input**, read it, and return an index. You need `while`

loops with two changing variables (Lab 01), integer division `//` (Lab 01), and recursion with a default argument (Lecture 03).

0.3 Read the skeleton

Open `dsa/searching.py`. There is no class this week, so there is nothing **given** except the module docstring, `__all__`, and the eight docstrings. All eight function bodies are **yours**. Read every docstring before you write a line: each one states the contract and the target cost.

Function	Returns	Input must be
<code>linear_search</code>	index of the first match, or <code>-1</code>	anything
<code>binary_search</code>	an index of target, or <code>-1</code>	sorted
<code>binary_search_recursive</code>	the same; takes <code>lo=0</code> , <code>hi=None</code> too	sorted
<code>lower_bound</code>	first <code>i</code> with <code>values[i] >=</code> target; may be <code>len(values)</code>	sorted
<code>upper_bound</code>	first <code>i</code> with <code>values[i] ></code> target; may be <code>len(values)</code>	sorted
<code>jump_search</code> , <code>exponential_search</code> , <code>interpolation_search</code>	index, or <code>-1</code>	sorted

0.4 The storage rule, this week

The storage rule says a `dsa/` **structure** keeps its data only in the course `Array`, in nodes, or in another `dsa/` structure. These are functions, not structures: they store nothing, and a list as input is allowed (Lecture 08, “This Week”). Two rules follow from the point of the week instead:

- **Do not copy the input.** No `values[a:b]`, no `sorted(values)`, no `list(values)`. A copy is $O(n)$ and destroys the $O(\log n)$ you are writing.
- **Do not call the library to do the search.** No `in`, `values.index`, or `bisect` inside `dsa/searching.py`. You will use `bisect` in Part 8 — as a check, after your own bounds work.

Part 1 — `linear_search`: the baseline

Lecture 08, “Linear Search”. Look at every element in order; return the index of the **first** one equal to target; if the loop ends, return `-1`. It is the only function this week that must work on unsorted data.

Write it. A `for i in range(len(values))` loop, one comparison, and the return `-1` **after** the loop. Read each element as `values[i]`: every search this week touches its input only through `len(values)` and `values[i]`, and that is exactly what Part 7 will count.

Test it.

```
pytest tests/test_searching.py -v -k "linear_search and not agrees"
```

This runs two tests: `test_linear_search_works_on_unsorted_data` and `test_linear_search_returns_`
which searches `[4, 7, 7, 1, 7]` for 7 and expects 1. Do **not** use plain `-k linear`: it selects
8 tests, because `test_agrees_with_linear_search_on_random_data` (five of them) and
`test binary search does not scan linearly` contain the word too.

When it fails.

- `assert -1 == 2` for `[5, 2, 9]` and `9` (and `assert -1 == 1` in the first-match test): the return `-1` is **inside** the loop, so the function gives up after looking at one element. Dedent it.
- `assert 4 == 1` in `test_linear_search_returns_the_first_match`: the loop remembers `i` and keeps going, so it returns the **last** match. Return as soon as you find one.

Part 2 — binary search: halve what is left

Lecture 08, “The algorithm, and the promise it keeps”. The lecture gives the steps; your job is to make every + 1 and <= agree with one decision: **lo and hi are both inclusive**, so the part still in play is `values[lo..hi]`. The docstring of `binary_search` names the four classic bugs from the lecture; Part 2.5 shows what each one looks like when you run the tests.

Invariant: if target is anywhere in values, it is in values[lo..hi].

2.1 Draw it

On paper, before any code. Use the lecture's table format — one row per turn of the loop, and a last row showing `lo > hi` for a miss:

```
lo    hi    mid    values[mid]    decision
```

You can also draw each step. highlight marks mid; done marks what has been discarded (paste into the notebook `notebooks/08-searching.ipynb`):

```
from viz.draw import draw_array
A = [4, 9, 13, 20, 26, 31, 37, 44]
draw_array(A, highlight=3, title="lo=0 hi=7 mid=3")
draw_array(A, highlight=5, done=[0, 1, 2, 3], title="lo=4 hi=7 mid=5")
```

Checkpoint 1. With $A = [4, 9, 13, 20, 26, 31, 37, 44]$, trace `binary_search(A, 37)` and `binary_search(A, 10)` as tables of `Lo`, `hi`, `mid` and `values[mid]`. How many comparisons does each make? For the miss, what is `Lo` when the loop ends, and what does that number mean?

2.2 Test sizes 0, 1 and 2 by hand

Lecture 08, “Easy to state, hard to get right”: nearly every binary-search bug shows up on a list of length 0, 1 or 2. Do these **before** you run pytest — they take two minutes and they find the bug while you still remember why you wrote the line.

Checkpoint 2. For a correct `binary_search`, give the return value and the number of times the loop body runs:

- (a) `binary_search([], 5)`
- (b) `binary_search([7], 7)`
- (c) `binary_search([7], 3)`
- (d) `binary_search([3, 8], 3)`
- (e) `binary_search([3, 8], 8)`
- (f) `binary_search([3, 8], 5)`

Then: if the loop condition were `lo < hi` instead of `lo <= hi`, which of the six answers would change?

2.3 Write it

1. Set `lo` to the first index and `hi` to the **last** index (not the length).
2. While the range values `[lo..hi]` is non-empty:
 - compute `mid` with integer division;
 - read `values[mid]` **once** into a local variable;
 - equal: return `mid`;
 - too small: everything up to and including `mid` is ruled out — move `lo` past it;
 - too large: everything from `mid` on is ruled out — move `hi` below it.
3. The range is empty: return `-1`.

Hints:

- Both updates must **move past** `mid`. If an update can leave `lo` and `hi` unchanged, the loop never ends.
- Write `mid = lo + (hi - lo) // 2` or `(lo + hi) // 2`. In Python both are correct; in Java and C only the first is safe (Lecture 08, the Bloch story). Say which you chose, and why, when the TA asks.
- Reading `values[mid]` once per turn is not only tidy: in Part 7 you will count reads, and a function that reads it twice will look twice as expensive.

2.4 Test it

```
pytest tests/test_searching.py -v -k "binary_search and not recursive"
```

This selects 11 tests: the four cases of `test_finds_target_in_sorted_list` (targets 1, 5, 9 and the missing 4 in `[1, 3, 5, 7, 9]`), `test_handles_empty_and_single_element`, the three cases of `test_target_outside_the_values` (0, 10 and 99 in the same list — below the first value, just past the last, and far past it), `test_finds_first_and_last_element`, `test_agrees_with_linear_search_on_random_data` — all with `[binary_search]` in

the name — and `test_binary_search_does_not_scan_linearly`, which searches a million integers and is slow if your code scans. The `and not recursive` part matters: `binary_search_recursive` contains `binary_search`.

2.5 When it fails

Bug	What you see	Fix
<code>mid = (lo + hi) / 2</code>	every test fails with <code>TypeError: list indices must be integers or slices, not float</code>	// — an index is an <code>int</code>
no return <code>-1</code> after the loop	<code>assert None == -1</code> in 6 tests, for example on <code>([], 1)</code> — the function fell off the end	add the final return <code>-1</code>
<code>hi = len(values)</code> with <code>lo <= hi</code>	<code>IndexError: list index out of range</code> in 4 tests: empty/single, the outside targets 10 and 99, and random data	inclusive <code>hi</code> is the last index, <code>len(values) - 1</code>
<code>lo = mid</code> or <code>hi = mid</code>	no failure message at all: the run stops at one test and never finishes — for <code>lo = mid</code> , the case that searches for 9	press Ctrl+C ; the traceback shows the while line. Use <code>mid + 1</code> and <code>mid - 1</code>

The last one is worth tracing once. With `lo = mid`, searching `[3, 8]` for 8: `lo = 0`, `hi = 1`, `mid = 0`, `3 < 8`, so `lo = mid = 0` — and nothing has changed. The next turn is identical, for ever. With `hi = mid`, searching `[42]` for 7 sticks at `lo = hi = 0` in the same way.

Part 3 — Break it: `lo < hi`

Homework 8, item 4, done in the lab where you can ask about it. Your `binary_search` passes its 11 tests. Now change **one character**: `lo <= hi` becomes `lo < hi`. Leave everything else alone.

Checkpoint 3. Before you run anything:

- With `lo < hi`, the loop stops as soon as the range has **one** element left, without looking at it. Which **positions** of a list can the broken search never find? Is there one it misses for every $n \geq 1$?
- Of the 11 tests selected in Part 2.4, which fail? Write down their names.
- The random-data test and the three outside-the-values tests pass. Why do they not catch

this bug?

Now run it and compare with your prediction:

```
pytest tests/test_searching.py -v -k "binary_search and not recursive"
```

Read each failing assertion aloud. `assert -1 == 4` means your function returned `-1` where the test expected `4`: it **missed** a value that was there. A bug that turns hits into misses is the worst kind — no exception, no crash, just a wrong answer.

Then **put the = back** and run the command again: 11 passed. If you are not sure you restored exactly what you had, `git diff dsa/searching.py` shows every line you have changed since your last commit.

What to take from this: a test suite is a set of examples, and the bug passes every example that never needs the last one-element range. The empty list, the single element, and the first and last positions are in the tests for exactly this reason (Lecture 08, “Easy to state, hard to get right”).

Part 4 — binary_search_recursive

Lecture 08, “Recursive binary search”. Same contract, same inclusive range — but instead of a loop that moves `lo` or `hi`, a **call** on the half that can still hold the target.

4.1 Draw it

Draw the calls as a column of frames, one per call, each with its own `lo` and `hi` — the call-stack picture from Lecture 03. The deepest frame is the one that returns first.

Checkpoint 4. A student writes this “elegant” version:

```
def bsr_slice(values, target):
    if not values:
        return -1
    mid = len(values) // 2
    if values[mid] == target:
        return mid
    if values[mid] < target:
        return bsr_slice(values[mid + 1:], target)
    return bsr_slice(values[:mid], target)
```

- What do `bsr_slice([1, 3, 5, 7, 9], 9)` and `bsr_slice([1, 3, 5, 7, 9], 1)` return? Which half is safe to slice, and which is not?
- For your correct `binary_search_recursive(List(range(1000)), -1)`, how many calls are on the stack at the deepest point, counting the last call whose range is empty?

4.2 Write it

1. `hi=None` is the usual way to write a default that depends on another argument. If `hi` is `None`, set it to the last index. Write `is None`, not `if not hi` — see the bugs below.
2. Base case: the range is empty (`lo > hi`) — return `-1`.
3. Compute `mid` and read `values[mid]` once.
4. Equal: return `mid`. Otherwise **return** the result of a call on `lo..mid - 1` or `mid + 1..hi`, passing the same values — never a slice.

4.3 Test it

```
pytest tests/test_searching.py -v -k recursive
```

Ten tests: the four found/missing cases, empty and single, the three targets outside the values, first and last, and random data — each with `[binary_search_recursive]` in its name.

4.4 When it fails

Bug	What you see	Fix
slicing, as in Checkpoint 4	<code>assert 1 == 4</code> for <code>([1, 3, 5, 7, 9], 9)</code> and <code>assert 1 == (50 - 1)</code> in the first/last test	pass <code>lo</code> and <code>hi</code> ; never slice
a recursive call without return in front	<code>assert None == 0</code> , <code>assert None == 4</code> , ... — 9 of the 10 tests fail	the result of the inner call must be returned by every outer call
<code>if not hi:</code> instead of <code>if hi is None:</code>	only the random-data test fails, with <code>RecursionError: maximum recursion depth exceeded</code>	<code>hi = 0</code> is a real range end, and not <code>0</code> is <code>True</code> : the range silently resets to the whole list. Test for <code>None</code> with <code>is</code>

The third bug is the nasty one: searching `[1, 3, 5, 7, 9]` for 1 or 0 works, but for 2 the search narrows to `lo = 1`, `hi = 0`, which is empty and correct — except that `hi = 0` is falsy, so the call starts again from `hi = 4`, and recurses until Python stops it.

Part 5 — lower_bound and upper_bound: one loop, one comparison

Lecture 08, “Bounds: Searching for Positions”. A bound answers “**where**”, and always with a position: the insertion point, or the start or end of a run of duplicates. Neither function ever returns `-1`; both may return `len(values)`.

5.1 The half-open range

The lecture writes both with a **half-open** range $[lo, hi)$: lo is a candidate, hi is **not**. Start with $lo = 0, hi = \text{len}(\text{values})$ — so $\text{len}(\text{values})$ is a possible answer — and loop while $lo < hi$, that is, while the range still has a candidate.

Notice what changed from Part 2. There, $lo < hi$ was a bug; here it is correct, because hi means something different. The loop condition follows from what lo and hi mean — it is not a style choice.

The invariant for `lower_bound`: everything **left of** lo is $< \text{target}$, and everything **from** hi on is $\geq \text{target}$. When the range closes, $lo == hi$ is the boundary between the two — the answer.

For `upper_bound`, replace $< \text{target}$ by $\leq \text{target}$ and $\geq \text{target}$ by $> \text{target}$. That is the only difference between the two functions.

5.2 Draw it

Take $v = [1, 2, 2, 2, 5]$ and target 2. Mark each element **L** ("goes left of the answer") or **R**, for each function:

index	0	1	2	3	4	
v	1	2	2	2	5	
lower_bound: $v[i] < 2$?			L	R	R	R
upper_bound: $v[i] \leq 2$?			L	L	L	R

-> first R at 1
-> first R at 4

Both functions are the same binary search for the **first R**. Only the test that decides L or R differs.

Checkpoint 5. $v = [3, 3, 5, 8, 8, 8, 9]$. Give:

- (a) `Lower_bound(v, 8)` and `upper_bound(v, 8)`, and how many 8s there are;
- (b) `Lower_bound(v, 4)` and `upper_bound(v, 4)`;
- (c) `Lower_bound(v, 10)` and `upper_bound(v, 0)`;
- (d) the number of 3s, computed with the two bounds.

5.3 Write it

Write `lower_bound` first:

1. lo, hi = the half-open range of the whole list.
2. While the range is not empty: take mid . If $\text{values}[mid]$ is an **L**, the answer is to its right — move lo past mid . Otherwise mid is an **R**, so the answer is mid or something left of it: hi becomes mid , not $mid - 1$. (Everything from hi on is R — that is the invariant — and mid has just been proved R.)
3. Return lo . Check it by hand on $[], [2]$ and $[1, 2]$ with target 2: the answers are 0, 0 and 1.

Then copy it to `upper_bound` and change **one comparison**. If you find yourself changing anything else, stop and re-read 5.1.

Why $hi = mid$ is safe here when it hung the loop in Part 2: with a half-open range, mid is always

strictly less than hi , so $hi = mid$ always shrinks the range. Try $lo = 0$, $hi = 1$: $mid = 0$, and either update makes the range empty.

5.4 Test it

```
pytest tests/test_searching.py -v -k bound
```

Three tests: `test_lower_bound`, `test_upper_bound` and `test_bounds_count_duplicates`, which checks that `upper_bound(v, 2) - lower_bound(v, 2)` is 4 on `[1, 2, 2, 2, 2, 5]`.

5.5 When it fails

Bug	What you see	Fix
<code>hi = len(values) - 1</code> (inclusive habit)	<code>assert 4 == 5</code> for <code>lower_bound([1, 2, 2, 2, 5], 6)</code> — the answer <code>len(values)</code> is unreachable	<code>hi = len(values)</code>
<code>return -1</code> when the answer is <code>len(values)</code>	<code>assert -1 == 5</code>	a bound is a position; <code>len(values)</code> is a valid one
<code>upper_bound</code> with <code><</code> instead of <code><=</code>	<code>assert 1 == 4</code> in the upper-bound test, and <code>assert 0 == 4</code> in the duplicates test	this is <code>lower_bound</code> again: a 2 must count as L for <code>upper_bound</code>
<code>while lo <= hi</code> with <code>hi = mid</code>	the run stops and never finishes	half-open means <code>lo < hi</code>

Part 6 — Jump, exponential and interpolation search

Lecture 08, “Three More Searches”. Each of these is a short function with one idea and two or three edge cases. The same ten tests run on each of them, so a working `binary_search` is your model for what “correct” looks like.

6.1 Draw it

Use one list for all three: `B = [0, 5, 10, 15, 20, 25, 30, 35, 40, 45]` ($n = 10$), target 30, which is at index 6.

Checkpoint 6. For each search, list the **indices** it reads, in order, until it returns 6:

- (a) `jump_search(B, 30)` — the step is `math.isqrt(10)`;
- (b) `exponential_search(B, 30)` — slot 0 first, then the bounds 1, 2, 4, ..., then a binary search between the last two bounds;
- (c) `interpolation_search(B, 30)` — it reads `B[lo]`, `B[hi]`, then the guess.

6.2 jump_search

1. Empty list: return -1 . Otherwise `step = math.isqrt(n)`, at least 1.
2. Jump block by block: while there is a **next** block and the **last** element of the current block is smaller than the target, move to the next block.
3. Walk forward inside the block you stopped in — but not past `n`: the last block may be shorter than `step`. Return the index if you find the target; return -1 as soon as you pass a value larger than it, or when the block ends.

```
pytest tests/test_searching.py -v -k jump
```

Bug	What you see	Fix
<code>step = n ** 0.5</code>	<code>TypeError: list indices must be integers or slices, not float</code>	<code>math.isqrt(n)</code> returns an <code>int</code>
the walk runs to <code>prev + step</code> with no <code>min(..., n)</code>	<code>IndexError: list index out of range</code> in <code>test_target_outside_the_values</code> for 10 and 99 — the only 2 of the 10 tests that walk off the end of the short last block	cap the walk at <code>n</code>

6.3 exponential_search

1. Empty list: -1 . If slot 0 holds the target, return 0 — the doubling starts at 1.
2. `bound = 1`; while `bound < n` and `values[bound] < target`, double it.
3. The target, if present, is between `bound // 2` and `bound` — but `bound` may be past the end, so cap it at `n - 1`. Binary search that range.

For step 3 you may reuse your own code: `binary_search_recursive` already takes `lo` and `hi`, so one call does it. That is a good reason it takes them.

```
pytest tests/test_searching.py -v -k exponential
```

Bug	What you see	Fix
<code>bound = 0</code> to start	the run stops at the case that searches for 5, and never finishes: <code>0 * 2</code> is 0	start at 1; check slot 0 separately
no <code>bound < n</code> in the doubling loop	<code>IndexError: list index out of range</code> in 5 tests, including [42] searched for 7 and the outside targets 10 and 99	test <code>bound < n</code> first : and stops before the read

Bug	What you see	Fix
hi = bound, not capped	IndexError: list index out of range in 4 tests: the outside targets 10 and 99, first/last, and random data	min(bound, n - 1)

6.4 interpolation_search

1. lo, hi inclusive, as in Part 2. While lo <= hi: read values[lo] and values[hi] once each.
2. **Trap 1 — outside the range.** If the target is below values[lo] or above values[hi], it is not there: return -1.
3. **Trap 2 — equal ends.** If values[hi] == values[lo], the formula would divide by zero. Every value in the range is the same: compare one of them with the target and return.
4. **Trap 3 — integer division.** Compute the guess with the lecture's formula, using //, so that pos is an int.
5. Then as in binary search, with pos in place of mid.

```
pytest tests/test_searching.py -v -k interpolation
```

Bug	What you see	Fix
no equal-ends check	ZeroDivisionError: integer division or modulo by zero in the empty/single test — a one-element range has equal ends	trap 2
no out-of-range check	the run stops at the missing-target case (4 in [1, 3, 5, 7, 9]) and never finishes	trap 1 — see below
/ in the formula	TypeError: list indices must be integers or slices, not float in 6 of the 10 tests	trap 3

The missing range check does not crash; it hangs. Searching [1, 3, 5, 7, 9] for 4: the first guess is 1 (value 3), so lo = 2. Now the range is [5, 7, 9], the target 4 is below it, and the formula gives $2 + (4 - 5) * 2 // 4 = 1$ — a guess **left of lo**. Value 3 is too small, so lo = pos + 1 = 2 again. Nothing changes, for ever. The check in trap 1 is what keeps every guess inside [lo, hi].

Part 7 — Count the reads

Lecture 08, “Measured”, right-hand panel. A timing depends on your machine; a **count** of how many elements a search reads does not. You will count them with a class of your own that looks like a list to the search.

7.1 Write CountingReads

This is not graded, and it does not go in `dsa/`: write it in the scratch space at the end of `notebooks/08-searching.ipynb`, or in a file of your own outside `dsa/`. The contract:

- `CountingReads(values)` keeps the list and a counter reads, starting at 0;
- `len(c)` is the length of the list;
- `c[i]` adds 1 to reads and returns `values[i]`.

Three methods: `__init__`, `__len__` and `__getitem__` (Lab 03, “Special methods”). Your searches use only `len(values)` and `values[i]`, so they cannot tell a `CountingReads` from a list. Check it first:

```
>>> c = CountingReads([10, 20, 30])
>>> len(c), c[1], c.reads
(3, 20, 1)
```

(`tools/figures_l08.py` has the course’s version, also called `CountingReads`. Look at it after writing yours.)

Checkpoint 7. With $V = \text{List}(\text{range}(1024))$ and a fresh `CountingReads(V)` for each call, predict reads after:

- `linear_search(c, -1)`
- `binary_search(c, -1)`
- `binary_search(c, 1024)`
- `interpolation_search(c, 500)`
- and `interpolation_search` for 500 on the **skewed** list `List(range(1023)) + [10**9]` — 1,023 evenly spread values and one huge one.

7.2 All the searches, side by side

```
import random
from dsa import searching

SEARCHES = [searching.linear_search, searching.binary_search,
             searching.binary_search_recursive, searching.jump_search,
             searching.exponential_search, searching.interpolation_search]

def average_reads(search, values, targets):
    counted = CountingReads(values)
    for t in targets:
```

```

        assert search(counted, t) == values.index(t)
    return counted.reads / len(targets)

rng = random.Random(8)
for n in [1024, 65536]:
    values = sorted(rng.sample(range(10 * n), n))
    targets = [rng.choice(values) for _ in range(100)]
    print(f'n = {n}')
    for search in SEARCHES:
        reads = average_reads(search, values, targets)
        print(f' {search.__name__:24} {reads:10.1f}')

```

With the reference solutions, the averages come out at about:

Search	n = 1,024	n = 65,536	Shape
linear	522	33,000	$n/2$
binary (loop or recursive)	9.0	14.6	$\log_2 n$
jump	34	243	about $2\sqrt{n}$ at worst, $\sqrt{n}$ on average
exponential	17.5	30	about twice binary
interpolation	8.5	11.6	nearly flat: $\log \log n$, three reads a step

Yours may differ by a small constant — a binary search that reads `values[mid]` twice per turn shows twice the reads — but the **shape** must be the same: multiply n by 64 and linear grows 64 times, jump about 8 times ($\sqrt{64}$), and binary adds about 6 reads ($\log_2 64$).

7.3 Interpolation on skewed data

```

n = 1024
even = list(range(n))
skewed = list(range(n - 1)) + [10 ** 9]
targets = random.Random(1).sample(range(n - 1), 50)
for name, values in [('even', even), ('skewed', skewed)]:
    for search in (searching.binary_search, searching.interpolation_search):
        reads = average_reads(search, values, targets)
        print(f'{name:7} {search.__name__:22} {reads:8.1f}')

```

On the even list interpolation needs 3 reads — one guess, straight to the answer — against about 9 for binary search. On the skewed list binary search does not notice anything (about 9 again), while interpolation needs about 1,480: **worse than linear search**, because each of its one-slot steps reads three values. The one huge value at the end tilts the straight line, so every guess lands at the far left (Lecture 08, “Interpolation search: guess, don’t halve”). An average case that hides a bad worst case — this is it.

Part 8 — Check your bounds against bisect

Only now, with your own bounds passing their tests, meet the library versions (Lecture 08, “lower_bound and upper_bound”): `bisect.bisect_left` is `lower_bound` and `bisect.bisect_right` is `upper_bound`. Use them as an **oracle** — a second opinion — on many random lists **with duplicates**, which the graded tests only touch once:

```
import bisect, random
from dsa.searching import lower_bound, upper_bound

rng = random.Random(1)
for trial in range(10_000):
    values = sorted(rng.choices(range(20), k=rng.randint(0, 15)))
    target = rng.randint(-2, 22)
    case = (values, target)
    assert lower_bound(values, target) == bisect.bisect_left(values, target), case
    assert upper_bound(values, target) == bisect.bisect_right(values, target), case
print('10,000 random lists: all agree')
```

The targets run from -2 to 22 on purpose, so that “before everything” and “after everything” are both tried, and k can be 0 , so the empty list is too. If an assertion fails, the message is the smallest thing you need: the list and the target. Trace that one by hand.

This is also how you use `bisect` from now on: in your own programs, call the library; in `dsa/`, the point is that you can write it.

Part 9 — Measure it

Part 7 counted reads; this part times them, to see the lecture’s left-hand panel appear on your own machine. Run it in `notebooks/08-searching.ipynb`:

```
from viz.complexity import measure, plot_growth
from dsa.searching import linear_search, binary_search

sizes = [10_000, 20_000, 40_000, 80_000, 160_000, 320_000]
make = lambda n: (list(range(n)), -1)      # a miss: the worst case for both

linear = measure(lambda a: linear_search(*a), sizes, make)
binary = measure(lambda a: binary_search(*a), sizes, make)
plot_growth({'linear search': linear, 'binary search': binary},
            reference=['n', 'log n'])
```

`measure` builds each input outside the timing, and keeps the best of three runs. What to expect:

- **Linear** is a straight line through the origin: double n , double the time. It follows the dashed $O(n)$ reference.
- **Binary** looks flat — a few microseconds at every size. Its growth is real ($\log_2 320,000$ is about 18 against 13 for 10,000), but on this scale it is lost in timer noise, which is why Part 7 counted reads instead.

Add `loglog=True` to `plot_growth` and linear becomes a line of slope 1, while binary stays almost level. Your exact times will differ from anyone else's; the shapes will not.

Part 10 — Exercises at a glance

Function	Target cost	The trap	-k filter
<code>linear_search</code>	$O(n)$	<code>return -1</code> inside the loop; must return the first match	see Part 1 (2)
<code>binary_search</code>	$O(\log n)$, $O(1)$ space	inclusive <code>hi = n - 1</code> ; <code>lo <= hi</code> ; <code>mid ± 1</code> ; //	see Part 2.4 (11)
<code>binary_search_recursive</code>	$O(\log n)$, $O(\log n)$ stack	pass <code>lo</code> , <code>hi</code> , never slice; <code>hi</code> is <code>None</code> ; return the call	recursive (10)
<code>lower_bound</code>	$O(\log n)$	half-open <code>[0, n)</code> ; <code>lo < hi</code> ; <code>hi = mid</code> ; may return <code>n</code>	bound (3)
<code>upper_bound</code>	$O(\log n)$	the same loop with <code><=</code> in place of <code><</code>	bound (3)
<code>jump_search</code>	$O(\sqrt{n})$	<code>math.isqrt</code> ; the last, partial block	jump (10)
<code>exponential_search</code>	$O(\log i)$	slot 0 first; bound starts at 1; cap at <code>n - 1</code>	exponential (10)
<code>interpolation_search</code>	$O(\log \log n)$ average, $O(n)$ worst	target outside <code>[v[lo], v[hi]]</code> ; equal ends; //	interpolation (10)

All 56 at once:

```
pytest tests/test_searching.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in `dsa/searching.py` slices, sorts, or calls `in`, `index` or `bisect` — the tests cannot see all of those, but the TA will look.

Part 11 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week08-questions.md`, with answers in `week08-answers.md`. Do the questions before opening the answers.

1. **Part F — write the code**, in `practice/week08.py`: `search_rotated`, `integer_sqrt`, `find_peak`, `closest_value` and `min_capacity` (W8-C1 to W8-C5). Every one is a binary search in disguise, and the tests count your reads, so an $O(n)$ answer fails:

```
pytest tests/test_practice_week08.py -v
```

Hint for `closest_value`: it is your `lower_bound` and one comparison.

2. **W8-T1** — the trace from Homework 8, item 2: `binary_search` on `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` for 23 and for 60.
3. **W8-T3** — draw the recursive calls for `binary_search_recursive(V, 45)` and `(V, 7)`, and give the depth.
4. **W8-B2** — a binary search with `hi = mid`. You met it in Part 2.5; now find an input on which it hangs, without running it.
5. **W8-K3** — counting copies of `x` by finding one and walking outwards. What does it cost when every element is `x`? What do your bounds cost?

The worked solutions are in `solutions/dsa/searching.py` and `solutions/practice/week08.py` — for after you have tried. `pytest --solutions tests/test_practice_week08.py` runs the tests on them.

Part 12 — Bridge to Lecture 09: sorting

Every search in this lab except the first assumed a promise: **the input is sorted**. Lecture 09 is where sorted input comes from. Here is the first connection, using a function you already have.

`lower_bound(out, x)` is the position where `x` goes to keep `out` sorted. So inserting each item there, one at a time, **sorts** the items:

```
from dsa.searching import lower_bound

def sorted_by_inserting(items):
    out = []
    for x in items:
        out.insert(lower_bound(out, x), x)
        print(x, out)
    return out

>>> sorted_by_inserting([5, 2, 9, 1, 5, 6])
5 [5]
2 [2, 5]
9 [2, 5, 9]
1 [1, 2, 5, 9]
5 [1, 2, 5, 5, 9]
6 [1, 2, 5, 5, 6, 9]
[1, 2, 5, 5, 6, 9]
```

(out is an ordinary list, in a script of yours — not in `dsa/`.) Two questions to bring to Lecture 09:

- **What does it cost?** Finding the place is $O(\log n)$, but `insert` shifts every element after it — $O(n)$ (Lecture 02). For n items that is $O(n^2)$ in total, however clever the search. The search is not the bottleneck; the shifting is.
- **Where do equal items go?** With `lower_bound`, a new 5 goes **before** the 5s already there; with `upper_bound`, **after** them — so equal items keep their arrival order. That property is called **stability** (Lab 03, Part 1.2), and `tests/test_sorting.py` checks it: look for `test_insertion_sort_is_stable`.

This is the idea of **insertion sort**: grow a sorted prefix one item at a time. In `dsa/sorting.py` you will write `insertion_sort` by shifting elements yourself rather than calling `insert`, and its docstring already tells you the surprise: $O(n^2)$ in general, but $O(n)$ when the input is nearly sorted.

Summary

Idea	The one line to keep
Linear search	Needs no order; $O(n)$; return the first match.
Binary search	Sorted input; $O(\log n)$; <code>lo..hi</code> inclusive, <code>lo <= hi, mid ± 1</code> .
Invariant	If the target is anywhere, it is in <code>values[lo..hi]</code> .
Test by hand	Sizes 0, 1 and 2 find nearly every boundary bug.
<code>lo < hi</code> bug	Misses every position reached only as a one-element range — always the last.
Recursive	Pass <code>lo</code> and <code>hi</code> ; a slice costs $O(n)$ and loses the index.
Bounds	Half-open <code>[lo, hi)</code> , <code>lo < hi, hi = mid</code> ; <code>lower</code> and <code>upper</code> differ in <code><</code> against <code><=</code> .
Counting	<code>upper_bound - lower_bound</code> is the number of copies.
Jump / exponential	$O(\sqrt{n})$, forward only / $O(\log i)$, unknown length.
Interpolation	$O(\log \log n)$ on even data; $O(n)$ — worse than linear — on skewed data.
Measuring	Count reads with a class that has <code>__len__</code> and <code>__getitem__</code> .

Answers to the checkpoints

Checkpoint 1.

binary_search(A, 37)					binary_search(A, 10)				
lo	hi	mid	values[mid]		lo	hi	mid	values[mid]	
0	7	3	20	20 < 37: lo = 4	0	7	3	20	20 > 10: hi = 2
4	7	5	31	31 < 37: lo = 6	0	2	1	9	9 < 10: lo = 2
6	7	6	37	found: return 6	2	2	2	13	13 > 10: hi = 1
					2	1			lo > hi: return -1

Three comparisons each. The miss ends with `lo = 2`: the index where 10 would be inserted, between 9 and 13 — which is exactly `lower_bound(A, 10)`.

Checkpoint 2. (a) `-1`, 0 turns — `hi` starts at `-1`, so the loop never runs. (b) 0, 1 turn. (c) `-1`, 1 turn. (d) 0, 1 turn. (e) 1, 2 turns — `mid` is 0 first (`3 < 8`, so `lo = 1`), then 1. (f) `-1`, 2 turns — `lo = 1`, then `hi = 0`. With `lo < hi`, (b) and (e) become `-1`: both need a turn on a one-element range, and that turn never happens. The others do not change.

Checkpoint 3. (a) Every position that the search only reaches when the range has shrunk to a single element. The **last** position, `n - 1`, is always one of them: `mid` can only equal `hi` when `lo == hi`. For `[1, 3, 5, 7, 9]` the broken search cannot find index 1 or index 4. (b) Four fail: `test_finds_target_in_sorted_list[9-4-binary_search]` (9 is the last element), `test_handles_empty_and_single_element[binary_search]` (`[42]` for 42 is a one-element range from the start), `test_finds_first_and_last_element[binary_search]` (98, the last) and `test_binary_search_does_not_scan_linearly` (999,999, the last). Targets 1 and 5 are still found, and 4 is a miss, which is `-1` either way. (c) A search that never looks at a one-element range can only turn hits into misses. The three outside-the-values targets are misses, so `-1` is right anyway. The random-data test has only 2 of its 30 targets actually in their lists, both are found before the range shrinks to one, and for the 28 misses `-1` is right. In all, 4 fail and 7 pass.

Checkpoint 4. (a) `bsr_slice([1, 3, 5, 7, 9], 9)` returns **1**: the call on the slice `[7, 9]` finds 9 at index 1 of the slice, and nobody adds back the offset 3. `bsr_slice([1, 3, 5, 7, 9], 1)` returns 0, correctly: a left slice `values[:mid]` starts at index 0, so its indices are the original ones; a right slice does not. Both copy, so the whole search is $O(n)$ anyway. (b) 10. The ranges have 1000, 499, 249, 124, 61, 30, 14, 6, 2 and then 0 elements — nine halvings and one call for the empty range, which returns `-1`.

Checkpoint 5. (a) `lower_bound(v, 8) = 3`, `upper_bound(v, 8) = 6`: three 8s (`6 - 3`). (b) 2 and 2 — 4 is absent, so both bounds give its insertion point, and the count is 0. (c) `lower_bound(v, 10) = 7 = len(v)`: 10 goes after everything. `upper_bound(v, 0) = 0`: 0 goes before everything. Neither is `-1`. (d) `upper_bound(v, 3) - lower_bound(v, 3) = 2 - 0 = 2`.

Checkpoint 6. (a) Indices **2, 5, 8, 6**. The step is `isqrt(10) = 3`. The last elements of the blocks are 10 (index 2) and 25 (index 5), both smaller than 30; the next, 40 (index 8), is not, so the walk starts at index 6 and finds 30 at once. (b) Indices **0, 1, 2, 4, 8, 6**. Slot 0 holds 0; the bounds 1, 2 and 4 hold 5, 10 and 20, all smaller; 8 holds 40, so the binary search runs on 4..8, and its first `mid` is 6. (c) Indices **0, 9, 6**. `pos = 0 + (30 - 0) × (9 - 0) // (45 - 0) = 270 // 45 = 6`: on evenly spread values the first guess is exact.

Checkpoint 7. (a) **1024** — a miss reads everything. (b) **10**, and (c) **11**: going left, the range shrinks from 1,024 to 511, 255, ... (the middle is removed as well), so it empties after 10 reads; going right it shrinks to 512, 256, ..., 1, which takes one more — $\lfloor \log_2 1024 \rfloor + 1 = 11$. (d) **3**: `values[lo]`, `values[hi]` and a guess that is exactly right. (e) **1503**: the huge last value makes every guess land at `lo`, so each step rules out one element; reaching 500 takes 501 steps of three reads each. Linear search would read 501 values.